Perception Challenge

Implementation, Reproduction and Camera Operation

Ege Gursoy

30 October 2026

This report describes the perception implementation supplied on the `perception_challenge_full` branch. It covers the selected simulator estimator, the camera path, the input/output conventions and the experiments that determined the method choices. Commands are run from the team repository root. The full command reference is `docs/perception/RUNBOOK.md`.

1 What is supplied

The runtime is in `src/rcr/perception/challenge/`, with the three trained Microban landmark checkpoints and their matching schemas. The setup scripts install an isolated Python environment, fetch pinned public CAD, build M3T[1] and the seed-recovery binaries, and install the pinned climbing model. Inference loads its code and resources from this checkout and its configured cache. The original experiment workspace is not a runtime dependency.

Table 1: Available entrypoints and their requirements.

Command	Input	Output
`holds`	Stereo PNG and declared rig calibration	Hold JSON, scene envelope, fitting/recovery diagnostics
`reproduce`	Included saved stereo cases	Predictions and exact-reference comparison
`replay`	Saved RGB packets; measured joints for configured profiles	Robot/cup predictions and converted team pose
`capture`	Two RealSense color cameras	Timestamped RGB pairs and calibration metadata
`calibrate`	Captured chessboard pairs, measured square size	Intrinsics, residual distortion and stereo geometry
`live`	Color streams; measured mounting or synchronized joints	JSONL stream and atomic latest-result file

The supplied implementation has two uses. The offline hold estimator targets the released RCR image generator[2]. Camera operation estimates the robot in the wall frame and optionally maps the released cup geometry. These outputs use different reference points and must remain distinct.

[1] `https://github.com/DLR-RM/3DObjectTracking/tree/master/M3T`

[2] `https://github.com/jackybaltes/rcr-perception`

2 Installation and first checks

Use Linux x86-64 and Python 3.12. The learned robot estimator requires an NVIDIA GPU and CUDA 12.8-compatible runtime/development tools. The tested GPU is an RTX 5000 Ada with 32 GB. The default build target is compute capability 8.9; set `-cuda-arch` for another supported card.

```
git submodule update --init
python3.12 scripts/perception/setup.py --robot --cuda
source .venv-perception/bin/activate
python scripts/perception/run.py doctor --robot
```

Setup creates `.venv-perception` and a separate resource cache, normally `~/.cache/rcr-perception`. It preserves the team's base environment and third-party submodules. The native dependencies and GPU packages require several GB of storage and downloads. `RCR_PERCEPTION_HOME` selects a different cache; use the same value for setup and inference.

Subsequent commands use the activated environment. The native library dependencies have an explicit conda-forge lock; source commits and downloaded CAD hashes are pinned. A Docker recipe is supplied at `scripts/perception/Dockerfile`. The runtime and native builds have been tested directly. The Docker image itself was not built in this container, which has no Docker daemon. For camera capture and the measured-marker baseline without learned robot inference, use setup with `-cameras`.

The launcher checks ambient GPU load and free memory before inference. It serializes its own jobs and checks again every 30 minutes while an external workload occupies the GPU. It never stops another process.

3 Reproduce the selected hold estimator

3.1 Run the supplied cases

Five original stereo pairs, calibration files and saved reference hold records are included under `data/perception/holds/`.

```
python scripts/perception/run.py reproduce \
  --out outputs/perception/holds
```

The program completes inference before opening the expected hold dictionary. The new repository-backed run reproduced all 50 hold records exactly. This is a saved-case regression, separate from the earlier two blind 100-scene sets. For a new input:

```
python scripts/perception/run.py holds \
  --stereo stereo.png --calibration rig.json \
  --out outputs/perception/prediction
```

The PNG contains equal-width left/right panels. Use the provided calibration schema: image dimensions, K and `T_camera_scene` for each view. `holds.json` uses metres, intrinsic XYZ Euler radians, the released hold types and the CAD backplate anchor. It is not a socket-centre map.

3.2 Method and conditions

The estimator obtains color proposals, fits estimated supporting geometry, and renders the public hold CAD through both cameras. Multiple yaw and inclined-pose hypotheses are refined and selected using observed image residuals. The renderer includes the wall and estimated volumes, which matters when a tilted hold is partly hidden by them.

A conditional second stage identifies layouts generated by the public pseudo-random sampler. It searches the nonzero unsigned 32-bit seed range, requires several consistent pose/color agreements and a unique supported layout, then checks the candidate against RGB. No input seed, filename-derived seed, answer table or reference pose enters this stage. If recovery is unverified, the estimator retains the independent CAD result.

The exact result assumes an untouched native scene: ten Microban holds, zero to three volumes, public paint/CAD, the known 0.8×1.4 m wall and supplied rig calibration. Cube holds, arbitrary edits and physical photographs are outside this validation. Use `-cad-only` to disable recovery. Even that CAD profile retains assumptions about the source geometry and appearance. Exactness concerns hold dictionaries; full volume reconstruction is not claimed.

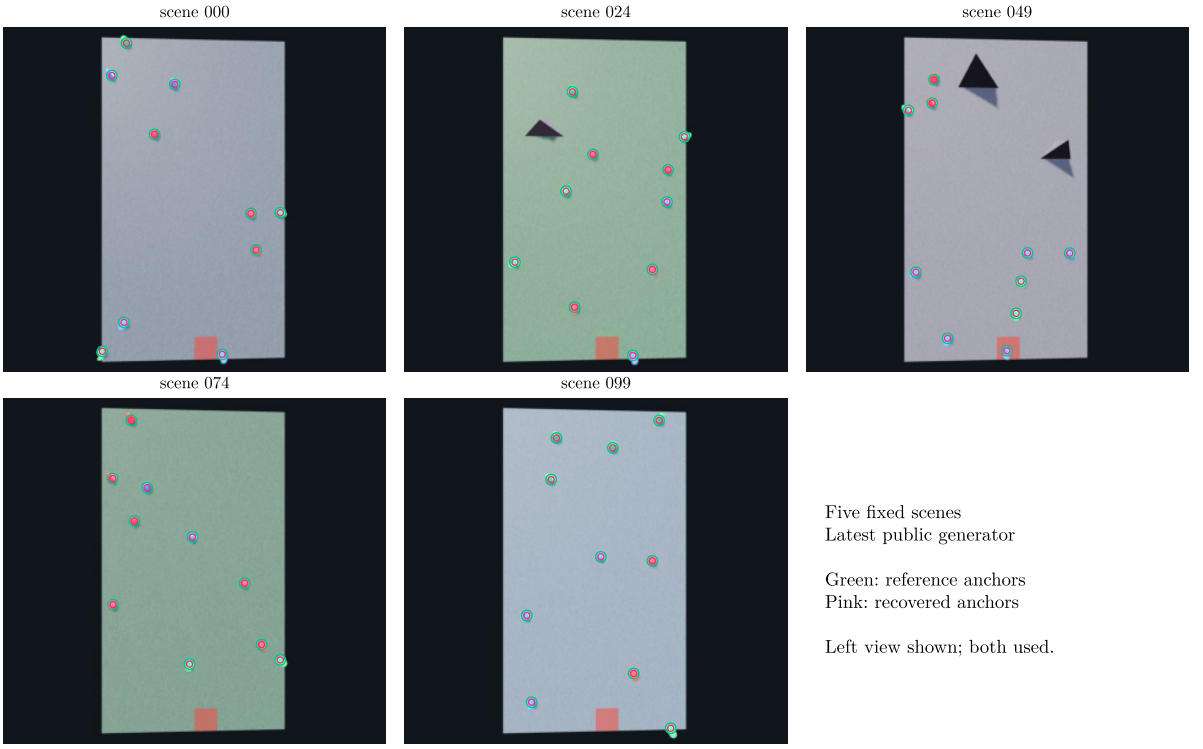


Figure 1: Five fixed examples from the latest-source set. Left views are shown; inference uses both. Green rings are source anchors and pink rings are recovered anchors. Numerical JSON equality establishes exactness.

Table 2: Earlier validation on two new 100-scene sets, 1,000 holds per set. Strict pose means position below 5 mm and rotation below 15°.

Set and method	Matched	Strict poses	Median position
Original source, independent CAD	999	998/1,000	0.104 mm
Original source, verified recovery	1,000	1,000/1,000	0
Latest source, independent CAD	998	995/1,000	0.108 mm
Latest source, verified recovery	1,000	1,000/1,000	0

The earlier recovery settings were frozen before these images and labels. These are local diagnostics and exact source-record comparisons. No official weighted competition score is inferred.

4 Reproduce robot pose and cup mapping

4.1 Known-joint and image-only profiles

With measured joints, the model configuration is known. The estimator solves for one six-dimensional trunk transform. A 20-point Microban network initializes the pose; M3T tracks 20 configured links; protected differentiable and exact CAD refinement reduce the remaining image error. If full-silhouette fitting rejects, a bounded partial-boundary search trims the largest mismatches while retaining the incoming estimate as a candidate.

```
python scripts/perception/run.py replay \
  --dataset data/perception/robot --profile max \
  --out outputs/perception/robot
python scripts/perception/verify_replays.py \
  --dataset data/perception/robot \
  --predictions outputs/perception/robot \
  --out outputs/perception/robot/verification.json
```

All five included cases passed both the root and endpoint checks in the repository-backed run. Root error ranged from 0.21 to 1.14 mm and 0.05 to 0.39°; worst endpoint errors ranged from 0.28 to 1.64 mm. References are loaded only by the scoring step. Failures remain in its denominator. Small numerical differences between GPU runs are expected; exact equality is checked only for the recovered hold records.

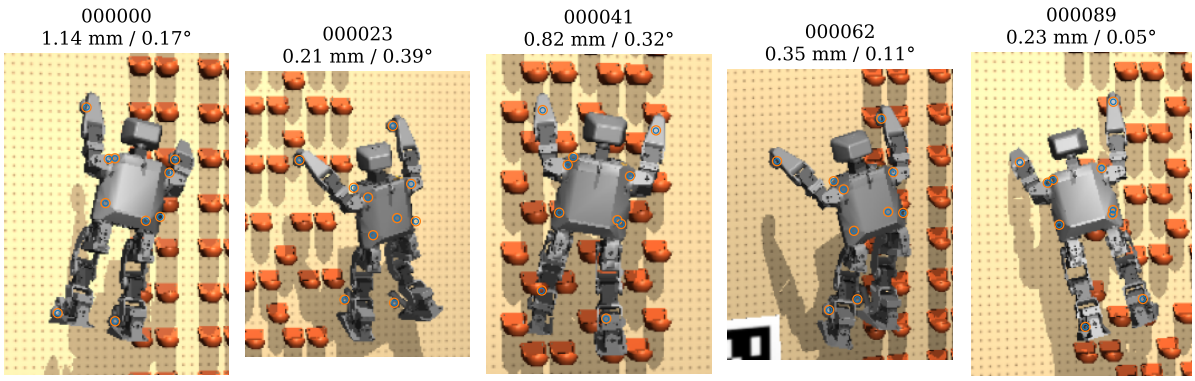


Figure 2: The five robot regression cases in this checkout. Orange rings show reference CAD landmarks and blue rings show landmarks transformed by the selected estimate. Titles give root translation and rotation errors. Both cameras contribute to inference; the left view is shown.

`max` is the selected static profile. `fast` retains the persistent M3T tracker; `accuracy` omits exact/boundary refinement. `trunk` combines two invariant-landmark networks and three crops before one geometric solve. It reads no joints, but supplies root pose only. It does not reconstruct articulation or support configured cup/robot occlusion reasoning.

4.2 Inputs and outputs

A saved packet identifies two RGB files, camera K, size, serial/ID and timestamp, a common clock, and all 19 named joint angles for configured profiles. The included packets illustrate the format. Image and joint measurements must be aligned within 15 ms; true camera/root poses, contact labels and native depth are not actor inputs.

`replay -task live -profile fast` adds dense cup mapping and conditional socket registration. The mapper uses RGB-supported components and the public cup/lattice geometry. The socket prior requires at least three non-collinear, unique and congruent correspondences; an 8

mm association gate is followed by a 1 mm fit check and bounded pose correction. It does not confirm seating or loading.

The repository-backed combined replay produced valid robot predictions and 72–86 mapped cup candidates across the five supplied scenes, with no map refresh errors. This is a regression check, not an independently scored new map benchmark. A map-refresh error remains explicit even when robot pose is valid.

The output retains the research-frame prediction and a converted `team_pose`. The research basis is x right, y up, z out; the team's basis is x right, y into the wall, z up. With a common origin, $p_{team} = (x, -z, y)$ and the same basis rotation must also multiply orientation. Quaternions are WXYZ. `team_api.robot_pose` returns the existing team `Pose`. `team_api.wall_model` uses mounting transforms and requires caller-supplied verified start contacts. It never converts a socket centre into a bolt midpoint by assumption or infers loaded contacts.

5 Operate two physical cameras

5.1 Capture and calibrate

List devices, choose serial order, and capture RGB pairs:

```
python scripts/perception/run.py capture --list
python scripts/perception/run.py capture \
  --serial LEFT_SERIAL --serial RIGHT_SERIAL \
  --width 1280 --height 720 --fps 30 --count 40 \
  --out outputs/camera_calibration
```

Only color streams are enabled. SDK projection rectifies color distortion; common-clock matching gates pairs at 15 ms. Timestamp matching does not establish simultaneous exposure. The pinned competition rules restrict external streams to RGB, so native depth/IR is not enabled.

Generate and print the chessboard at 100%, measure the printed square, and capture at least ten successful pairs with varied distance and tilt:

```
python3 scripts/perception/make_chessboard.py \
  --out outputs/chessboard.svg
python scripts/perception/run.py calibrate \
  --dataset outputs/camera_calibration --square 0.020 \
  --out outputs/calibrated_cameras
```

The vector board has nine by six internal corners and nominal 20 mm squares. Use the measured print size, not an assumed printer scale. The solver estimates intrinsics/residual distortion and relative stereo geometry from the captured, SDK-rectified RGB images. It reports accepted/rejected pairs and RMS residuals. Inspect coverage and repeat poor calibration. Wall pose is estimated separately.

Use `-camera-calibration` on subsequent capture/live runs. The runtime binds the measured calibration to the same serial order, image dimensions and source intrinsics. Applying it to another profile is rejected. It is not a raw-sensor distortion model.

5.2 Marker-based physical baseline

Mount marker ID 5 rigidly on the trunk. Measure the black-code size and the full `T_trunk_marker` transform. Copy `config/perception/marker.example.json`; replace its example geometry and set `measured` to true. The unmeasured template refuses to run.

Wall markers use DICT_4X4_50 IDs 1–4. Defaults match the released board; measured alternative marker corners can be supplied in the configuration. Each used view needs at least two wall markers. Do not use outer white-border size as black-code size.

```
python scripts/perception/run.py live \
  --serial LEFT_SERIAL --serial RIGHT_SERIAL \
  --estimator markers --marker-config config/perception/my_marker.json \
  --camera-calibration outputs/calibrated_cameras/calibration.json \
  --out outputs/perception/marker_live
```

The estimator considers planar PnP hypotheses and jointly refines the measured marker geometry against the camera observations. This path needs no learned model or encoder readings. It provides a calibration/reference baseline for checking the learned estimator; physical accuracy remains to be measured.

5.3 Learned root pose and live cups

For configured inference, record all 19 measured joint positions in JSONL with `clock=host_monotonic` and measurement timestamps mapped to the laptop clock. A Pi's independent monotonic clock and packet receipt times are not substitutes for aligned measurement time. The reader interpolates only nearby samples and rejects stale telemetry.

```
python scripts/perception/run.py live \
  --serial LEFT_SERIAL --serial RIGHT_SERIAL \
  --profile fast --task live --telemetry-log outputs/encoders.jsonl \
  --camera-calibration outputs/calibrated_cameras/calibration.json \
  --out outputs/perception/learned_live
```

Use `-calibration boundary` for the markerless wall-calibration profile, subject to its tested board/appearance assumptions. The private runtime uses the original Rhoban climbing geometry for regression. Current team main uses a cut-foot fork. Physical configured fitting must use the measured robot model through `RCR_MICROBAN_MODEL`; the runbook describes this override. The learned networks were trained in the released simulator. Their physical appearance transfer has not been evaluated. The cup map likewise depends on the released upright cup and measured mounting geometry; the simulator lattice does not establish physical bolt spacing or origin.

5.4 Freshness and controller use

`live` writes `predictions.jsonl` and atomically replaces `latest.json`. It reports acquisition time, processing time and age at output. Frames are selected from the recent synchronized pair buffer. Estimates older than `-max-output-age` (default 0.25 s) do not expose a usable team pose. Ctrl-C closes the owned camera pipelines and estimator.

Recorded simulator sequences have about 20 ms median full inference, but map refresh/reacquisition reached 3.01 s. The median is not a deadline. High-compute `max` is therefore most useful on saved captures; choosing a larger freshness limit for online use is an explicit controller decision.

6 Earlier methods and observed failures

FoundationStereo and S2M2 had mean target-depth errors of 9.614 and 9.872 mm. These errors matter on small holds; the smoother map did not reliably produce better poses. Accurate rendered-depth controls showed that the rigid-object adapters could work when depth was reliable. They did not establish performance with real depth cameras. Extra sampling, refinement or a larger network did not consistently improve the final output.

CAD fitting benefited from correcting wall/volume occlusion and evaluating multiple geometric hypotheses against both images. Native recovery then resolved source-layout ambiguity under its explicit generator assumptions. The original 1,000-scene sweep also showed why subset analysis

Table 3: Selected comparisons on the same historical 100-scene hold set, 1,000 holds. These are development results, distinct from the later blind recovery sets.

Method	Strict hold poses
FoundationStereo + FoundationPose	447/1,000
S2M2 + FoundationPose	278/1,000
FoundationStereo + SAM-6D	164/1,000
Combined RGB proposals + exact CAD	975/1,000
Learned proposals + exact CAD	976/1,000

matters: only 79 of 10,000 holds were volume-mounted, and their large orientation errors were hidden by the aggregate rotation median.

For the robot, FoundationPose with stereo passed 98 root checks but 62 endpoint checks on the original 100 cases. MegaPose passed 6 and 2. Both ran actual image inference. The configured keypoint/M3T/CAD estimator achieved more endpoint successes under those conditions. On two fresh 100-scene sets, selected max passed all root and endpoint checks, with median root errors 0.356/0.378 mm and 0.146/0.125°. These local thresholds are 5 mm/5° for the root and 5 mm for every endpoint; an official offline robot scorer has not been released.

An artificial 20% cutout initially caused full-silhouette refinement to discard usable poses. Preserving the incoming pose and adding trimmed boundary fitting raised successes to 100 root/99 endpoint on one set and 100/100 on another. Pixel noise and incorrect joints still cause endpoint failures. Lower image residual is insufficient when the model configuration or calibration is wrong.

7 Validation status and remaining work

The new checkout has reproduced the five supplied hold scenes and five robot poses with locally built native binaries and repository checkpoints. The calibration solver is tested on generated RGB chessboard pairs; the marker path is tested on an augmented RGB fixture with known mounting geometry. SDK import and enumeration work, but no cameras are attached here. These tests establish software behaviour and regression reproduction. They do not establish physical USB reliability, exposure synchronization, real-camera pose accuracy or perception-driven climbing performance.

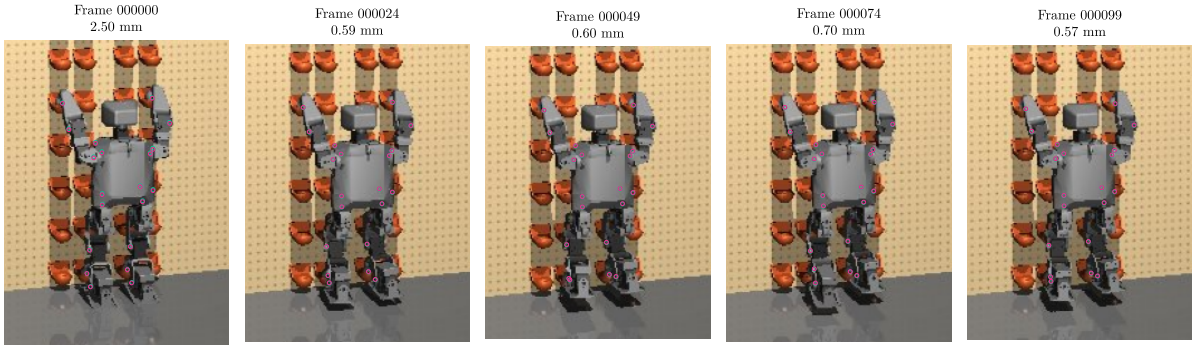


Figure 3: Earlier recorded simulator sequence, five selected frames. Projected model landmarks are overlaid on RGB. These correlated frames illustrate temporal tracking; they are not independent trials or physical-camera validation.

The original broad benchmark records remain in `docs/perception/metrics_snapshot.json`.

The current deployment checks and commands are recorded separately in `docs/perception/verification.json`. Reference data are opened after prediction, and failed cases remain counted. The setup and API preserve the team frame/types; they do not manufacture joint velocities, a verified start stance or loaded contacts.

Before controller use, measure the physical cameras, wall geometry and marker mount; verify timestamp alignment; inspect overlays and root/endpoint error against an independent reference; and test reacquisition delays. Those checks address the remaining difference between the recorded simulator results and the physical acquisition task.